

《人工智能通识》

第五讲: Function Call

——让大模型实时感知与交互

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾: 上一讲的 RAG, 三句话你还能说出来吗?

Q1 它解决什么? 模型“不知道”: 知识过时、太专、没出处

Q2 它怎么做? 切块建向量库 → 检索最相关片段 → 连同问题一起生成

Q3 它像什么? 开卷考试: 先翻手册找到那一页, 再作答

开卷考试有个前提: 答案得**已经写在纸上**。

可是“此刻反应器多少度”没写在任何文档里; “把数据导出来”更不是翻书能翻出来的。

RAG 解决了“不知道”, 但模型仍然**感知不到实时、也做不了事**。今天补上这一块。

举手投票

课堂互动①

上一页的三个问题，只靠“大模型 + RAG”能**可靠**答对几个？

A. 三个都能 B. 只有第 3 个 C. 一个都不能 D. 说不准
先举手，再听三位同学各说一句理由 (2 分钟)。

问题 1：现在多少度 实时数据，每秒变化

任何文档、任何训练语料里都没有

问题 2：导出成表 这是一个**动作**

翻再多书也导不出一张表

问题 3：蒸气压换算 物性可以查手册

但 Antoine 方程要算、单位要换，模型“心算”常出错

三个问题指向同一个缺口：模型需要一双**伸向外部世界的手**。

这双手，就是今天的主角——**Function Call (函数调用)**。

本讲学习目标

知识目标

说清 Function Call 是什么、解决哪三类问题; 讲出“声明函数 → 模型生成调用请求 → 程序执行 → 回填 → 回答”的回路; 知道它与 RAG、MCP、Agent 的分工。

能力目标

能用 JSON Schema 为化工小工具 (查物性、读 DCS 位号、单位换算) 写函数声明; 能判断一个请求该不该调函数、调哪个; 能给函数划分权限等级。

素养目标

理解“让 AI 做事”意味着责任转移到了设计者身上: 参数要校验、结果要核对、写操作要留人确认。

学习建议: 全程盯住一个问题——“这一步是模型在做, 还是程序在做?”

第一部分 · Function Call 到底是什么

Function Call, 直译就是“函数调用”。

先别急着问“模型怎么调函数”，先想清楚：什么是函数？

你每天都在无意识地用函数：在百度搜一个词，结果出来后点开一条链接——这个“点击”背后，就是一次函数调用：系统把你选的内容从数据库里取出来，再展示到网页上。

函数 = 一段封装好的程序：给它输入，它做一件确定的事，还你一个输出。

这些过程你看不到，但它们都在后台自动完成。

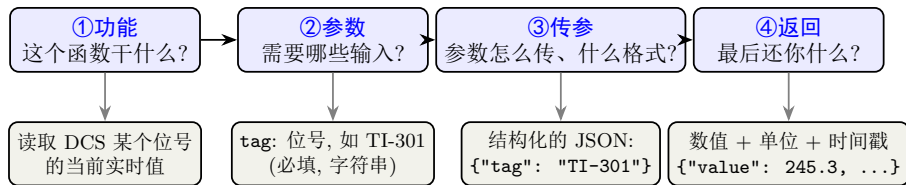
化工同学身边的“函数”：输入什么，还你什么

输入	函数	输出
位号 TI-301	<code>read_dcs_tag</code> (读 DCS 实时值)	245.3 °C (10:32:07 读数)
苯, 40 °C, 蒸气压	<code>query_property</code> (查物性数据库)	24.4 kPa
100 psi → kPa	<code>unit_convert</code> (单位换算)	689.5 kPa
CAS 71-43-2	<code>get_sds</code> (查安全数据表)	闪点 -11 °C, 易燃液体
进料 10 t/h, 转化率 85%	<code>mass_balance</code> (物料衡算)	各股物流的质量流量

Excel 里的 `=SUM()`、Aspen 里的物性方法、Python 里的 `def`, 全都是函数。

Q: 这些函数有一个共同点：给同样的输入，一定得到同样的输出。大模型有这个性质吗？

如果让你来设计一个函数, 先要回答四个问题



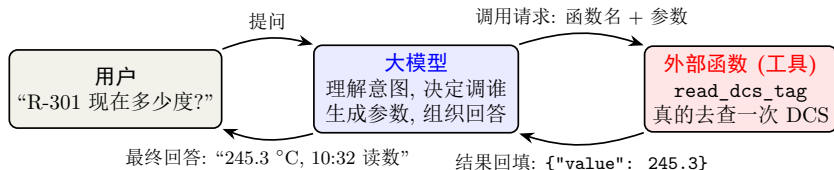
示例: `read_dcs_tag` —— 读 DCS 实时数据的函数

这就是函数的**基本要素**: 功能、参数、传参方式、返回值。

有编程经验的同学会发现: 这和写一个普通函数的思路一模一样。
不同之处只有一个——这一次, **决定“什么时候调、传什么参数”的不是你, 而是大模型。**

Function Call 的定义, 一句话

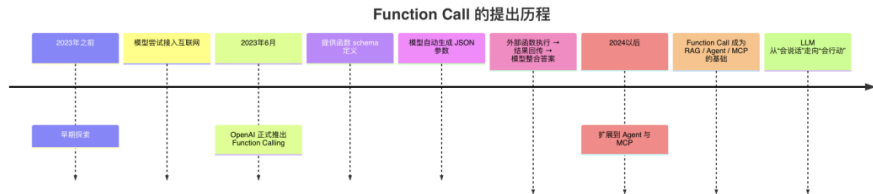
模型并不自己“知道”所有答案, 而是通过**调用外部函数 (工具)**把结果取回来,
再结合你的问题, 生成最终回答。



模型“会说话”, 函数“会做事”; Function Call 把两者接在一起

Q:模型怎么知道你写了哪些函数? $\Rightarrow$ 你得先把它们“**注册**”给模型 (第三部分详讲)。

它是怎么来的：从“胡编”到“会做事”

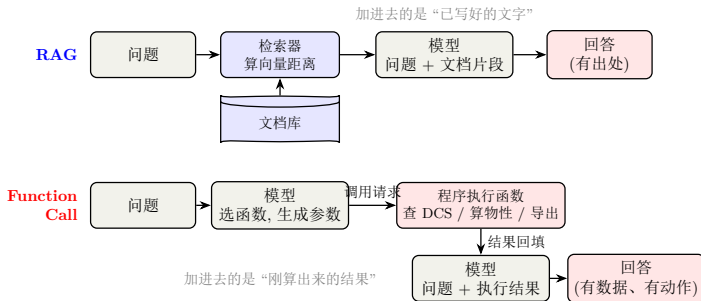


最初的 GPT-2、GPT-3 只能靠训练时的“记忆”作答，问到实时信息或外部操作就只能编。

2023 年 6 月 OpenAI 正式推出 Function Calling: 提供函数 schema 定义 → 模型自动生成 JSON 参数
→ 外部执行 → 结果回传。2024 年以后, 它成了 RAG、Agent、MCP 共同的地基。

配图: Function Call 的提出历程。如今 DeepSeek、通义千问、智谱等国产模型都支持同样的机制。

和 RAG 放在一起看：都往输入里加东西，加的不一样



RAG

加的是**已写好的文字**

谁决定：检索算法 (算向量距离)

Function Call

加的是**刚算出来的结果**

谁决定：模型自己 (理解意图, 生成参数)

两者并不冲突：实际系统里，“检索知识库”本身就常被做成一个可供模型调用的函数。

第二部分 · 为什么需要 Function Call: 模型的三类鸿沟

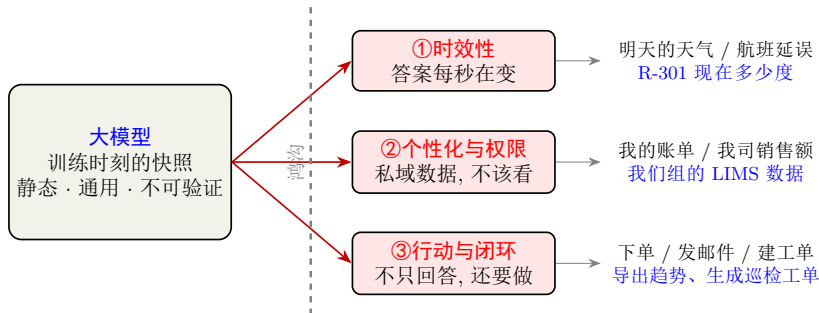
大模型很强, 但它的“脑子”来自**训练时刻的一张快照**:

- 静态的** 训练截止之后发生的事, 它一概不知
- 通用的** 公开语料里有的它才见过, 你们实验室的数据它没见过
- 不可验证的** 它说“245 度”, 你无法追问“这个数从哪来的”

一旦问题落到真实业务里, 就会立刻撞上三道鸿沟。

第一讲说过: 训练结束后, 参数就冻住了。推理时它只是在“回忆”。

三类鸿沟, 一张图



黑字: 生活里的例子 蓝字: 化工厂里的例子

时效性靠翻文档也解决不了; **权限**意味着模型“不该看”; **行动**要的不是
一段话, 而是一个结果。

鸿沟一：时效性——答案每秒都在变

	生活里	化工厂里
问 的 是 什 么	明天杭州的天气; 这班飞机延误没; 库存还剩多少	反应器当前温度; 储罐液位; 今天进厂原油的密度和硫含量
数据在哪	气象局、航司、仓库的实时系统	DCS、LIMS (实验室信息系统)、罐区计量系统
为 什 么 模 型 不 知 道	训练结束时这些事还没发生	DCS 每秒刷新, 任何语料都不可能收录

上一讲的 RAG 能帮上一点忙——如果有人**每天把报表存成文档**。

但“此刻”的值, 只能**此刻去问实时系统**。

Q:如果模型不承认自己不知道, 而是给你编了个“250 °C”, 在中控室里会发生什么?

鸿沟二与三：权限, 和“做事”

鸿沟二：个性化与权限

“我这个账号的账单”“我司本月销售额”“我们组上周的 GC 数据”。

这些是企业或个人的**私域数据**：模型本体既**看不到**，也**不该看到**——谁能查、查多少，必须由你的系统说了算。



鸿沟三：行动与闭环

不只是回答，还要去**做**：下单、发邮件、建工单、拉报表、写回数据库。

化工版：导出趋势曲线、生成巡检工单、**启动一次流程模拟**、把化验结果录入 LIMS。

单靠语言生成，这两道鸿沟都跨不过去：说得再像，邮件也没发出去，工单也没建起来。

模型需要的，是一条通向**真实系统**的、**受控**的通道。

Function Call 的价值: 把“会说话”的模型, 接上“会做事”的工具

开发者 先以函数声明定义可用能力: 名称、参数、返回值和**安全边界**

模型 理解意图后, 自动选择函数并生成**结构化参数**

应用 去调用真实世界的 API 或服务, 拿到**可验证**的结果

模型 再把结果组织成自然语言回复

从“静态记忆 + 概率式输出” $\Rightarrow$ “按需取数 + 安全执行 + 可控反馈”

答案**可追溯、可审计、可复现**, 幻觉降低; 企业现有系统 (数据库、DCS、LIMS、ERP、工单) 无缝纳入 AI 工作流。

“可追溯”的意思: 回答里的 245.3 °C, 能查到是哪个函数、什么时刻、从哪个位号读出来的。

判断题：该不该调函数？（每题举手，再揭晓）

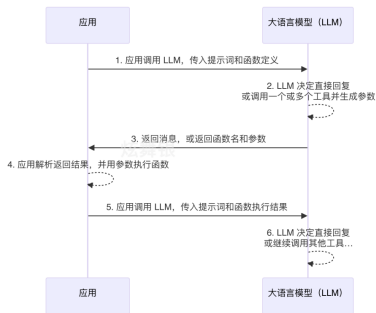
课堂互动②

1. “用大白话解释什么是催化裂化。”
2. “R-301 现在的入口温度是多少？”
3. “350 °F 等于多少摄氏度？”
4. “把这份实验小结发给我导师。”
5. “我们组上周三次 GC 分析的平均峰面积是多少？”

1. ✗ 通用知识，模型自己会答——不需要。
2. ✓ 实时数据（鸿沟一）——必须调 `read_dcs_tag`。
3. 两可：模型能心算，但在工艺计算里一位小数的错也不能犯——交给 `unit_convert` 更稳。
4. ✓ 这是动作（鸿沟三）——调 `send_email`，而且发之前要让你确认。
5. ✓ 私域数据（鸿沟二）——调 `query_lims`，且只能查你有权限的那部分。

判断标准：答案是否随时间变、是否只在你的系统里、是否要改变外部世界。

第三部分 · 核心机制：应用与模型之间的六步回路

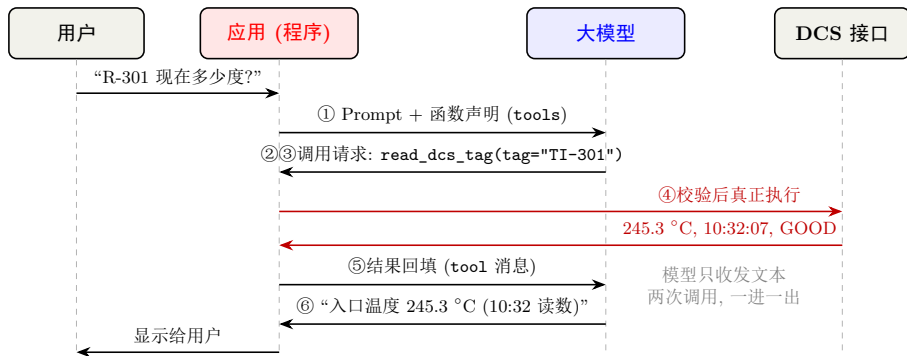


配图：应用 ↔ 大模型的时序图

1. 应用调用 LLM, 传入提示词 + 函数定义;
2. LLM 决策: 直接回复, 还是调用一个或多个工具并生成参数;
3. 返回消息, 或返回函数名和参数;
4. 应用解析返回结果, 执行函数;
5. 应用再次调用 LLM, 传入提示词 + 函数执行结果;
6. LLM 决定直接回复, 或继续调用其他工具。

注意箭头的方向：模型从头到尾只收发文本，真正“动手”的都在左边的应用那一侧。

换成化工场景,把六步画成一个闭环



问题“R-301 现在多少度”走完这一圈, 大约 1 ~ 3 秒; 用户看到的只是一句话, 中间发生了两次模型调用和一次 DCS 读取。

第 1 步: 应用发给模型的, 不只是问题

```
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4.1",
    messages=[ # Prompt: 对话内容 (系统提示词 + 用户问题)
        {
            "role": "system",
            "content": "你是一个智能助手, 可以通过调用函数来获取实时数"
        },
        {
            "role": "user",
            "content": "杭州今天天气如何?"
        }
    ],
    tools=[ # Tools: 函数声明 (采用 OpenAI JSON Schema 格式)
        {
            "type": "function",
            "function": {
                "name": "get_weather",
                "description": "查询某个城市在某一天的天气情况",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "location": {
                            "type": "string",
                            "description": "城市名称, 例如: 杭州"
                        },
                        "date": {
                            "type": "string",
                            "description": "日期, 格式 YYYY-MM-DD"
                        }
                    },
                    "required": ["location", "date"]
                }
            }
        }
    ],
)
print(response.choices[0].message)
```

配图: 一段 Python 调用代码。请求里有两块:

messages 对话内容: 系统提示词 + 用户问题 (第三讲的 Prompt)

tools 函数声明: 每个函数的 **name**、**description**、**parameters**

函数声明必须写成模型能读懂的统一格式——目前业界通用的是 **JSON Schema**。

图中用的是 OpenAI 的写法; DeepSeek、通义千问的 API 采用几乎相同的 **tools** 字段, 后面演示时会看到。

用 JSON Schema 描述一个化工函数: 逐行看

```
{
  "type": "function",
  "function": {
    "name": "read_dcs_tag",
    "description": " 读取 DCS 某位号
    当前实时值 (温度/压力/流量)",
    "parameters": {
      "type": "object",
      "properties": {
        "tag": {"type": "string",
          "description": " 位 号, 如
          TI-301"}
      },
      "required": ["tag"]
    }
  }
}
```

name	函数名, 程序按它去找真正的代码
description	给模型看的说明: 它据此决定该不该调
properties	每个参数的类型和含义
required	哪些参数必填

Schema 只是描述, 不含任何执行代码。

真正的 read_dcs_tag 函数体, 在你的程序里, 模型永远看不到。

第 2、3 步: 模型决策, 然后返回两种东西之一

情况 A: 直接回复

用户问“什么是加氢反应器”——可用函数里没有相关的, 也不需要, 模型照常生成一段文字。

情况 B: 返回调用请求

用户问“R-301 现在多少度”——模型输出:

```
"tool_calls": [{  
  "name": "read_dcs_tag",  
  "arguments": {"tag": "TI-301"}  
}]
```

请注意: 情况 B 里, **到此为止什么都还没发生**。

模型只是生成了一段格式正确的 JSON 文本——和它生成一首诗, 在原理上没有任何区别。

Q:模型是怎么从“R-301 入口温度”推出 `tag=TI-301` 的? 如果它推错了呢? (第四部分回答)

第 4、5 步: 程序执行, 把结果回填给模型

第 4 步 (程序做)

1. 解析 JSON, 取出函数名和参数;
2. 校验: 位号存在吗? 有权限读吗?
3. 真正调用 DCS 接口, 得到原始结果:

```
{ "tag": "TI-301", "value":  
  245.3,  
  "unit": "degC",  
  "timestamp": "2026-09-21  
10:32:07",  
  "quality": "GOOD" }
```

第 5 步 (程序做)

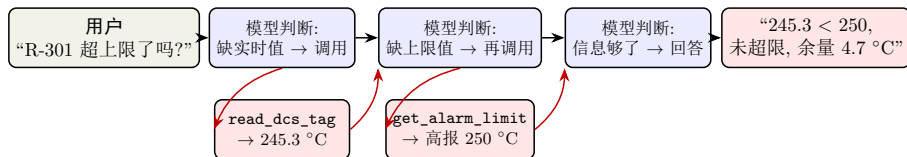
把这段原始结果, 作为一条新的消息追加到对话里 (角色是 tool), 连同之前的全部上下文再发给模型。

模型此时看到的对话是:

- ▶ 用户: R-301 现在多少度?
- ▶ 模型: 调用
 read_dcs_tag(TI-301)
- ▶ 工具: {value: 245.3, ...}

回填的结果也占上下文长度——一个返回 5 万行的函数, 会把窗口撑爆。

第 6 步: 生成最终回答——或者, 再调一次



每一次调用都是一个完整的“请求 → 执行 → 回填”小回路; 模型看到的上下文越来越长

问题稍复杂一点, 一次调用就不够了: “R-301 温度~~超上限了吗?~~” 需要先读实时值, 再查报警上限, 最后比较。

模型每拿到一个结果都会重新判断: 信息够了就回答; 不够, 就继续生成下一个调用请求。

“反复调用直到解决”这个循环, 正是第七讲 Agent 的雏形。

回到那个问题：模型是怎么“知道”你有哪些函数的？——注册

	方式一：写进 Prompt	方式二：tools 字段
怎么做	在系统提示词里用文字把每个函数的名字、参数、返回值描述一遍	调 API 时，把 JSON Schema 列表放进 tools 参数
模型怎么用	当成“候选工具”读一遍，靠理解力选	模型经过专门训练，会输出规范的 tool_calls
优缺点	灵活，但格式不稳、容易解析失败	结构化、可靠，是 目前的主流做法

“注册”**不是**把函数永久写进模型，而是**每一次**调用时，通过 Prompt + tools 把函数定义**临时**交给模型。

所以：换一个会话，函数定义得重新传；函数定义本身也在**消耗 token**（第二讲的账要算进来）。

现场演示：看一次真实的 Function Calling 请求

课堂互动③

打开 [DeepSeek API 文档 · Function Calling](#) 页面，跟着示例代码走一遍。
每人在纸上记下三件事 (5 分钟)，然后随机点 3 位同学核对：

1. `tools` 里声明了什么函数？它有几个参数，哪个是 `required`？
2. 模型第一次返回的内容里，有没有“温度是 24 度”这样的话？它返回的到底是什么？
3. 第二次请求比第一次多了哪一条消息？这条消息是谁写进去的？

备用：[阿里云百炼 · 通义千问 Function Calling 文档](#) (结构完全一样，可对照看)。

看懂这三点，六步回路就真的看懂了。

第三部分小结: 六步里, 谁负责什么

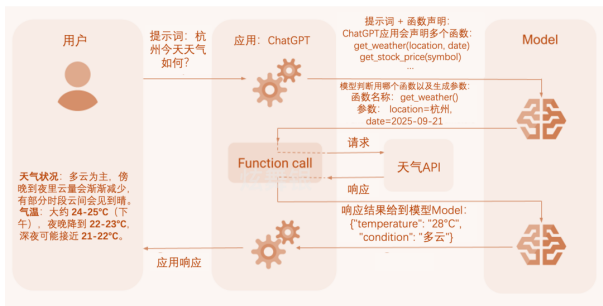
步	做什么	模型	程序
1	发送 Prompt + 函数声明 (tools)		✓
2	理解意图, 决定是否调用、调哪个、参数是什么	✓	
3	返回 tool_calls (函数名 + JSON 参数)	✓	
4	校验参数, 真正执行函数		✓
5	把执行结果作为 tool 消息回填		✓
6	基于结果生成回答, 或继续调用	✓	

模型只负责“**想**” (2、3、6), 程序负责“**做**” (1、4、5)。

Q:既然“做”的都在程序这边, 出了错该找谁? ⇒ 第四部分: 化工案例与工程细节

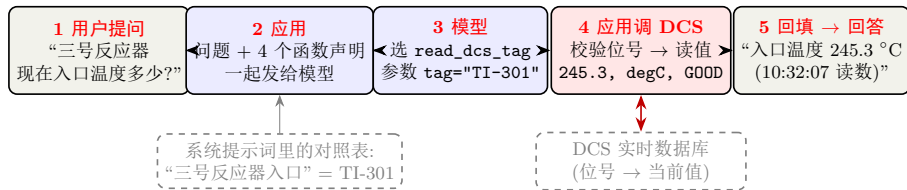
先看一个案例：“杭州今天天气如何？”

第四部分 · 化工案例与工程细节：校验、重试、权限



用户提问 → 应用把问题和函数声明一起交给模型 → 模型选中 `get_weather`, 生成参数 `location= 杭州, date=...`
→ 应用请求天气 API, 拿到 `{"temperature": "28°C", "condition": "多云"}`
→ 回填模型 → 生成给用户的回答。

把“查天气”换成“查反应器”：化工版完整走一遍



模型能把“三号反应器入口”翻译成 TI-301, 靠的是提示词或 RAG 提供的位号对照, 不是它自己“知道”

和天气案例的区别只有一处: 函数背后连的不是气象 API, 而是**你们厂的 DCS**。

回路一模一样——这正是 Function Call 的好处: 换一个函数, 就换了一个能力, 模型本身一行都不用改。

给“化工 AI 助手”准备的四个基础函数

函数名	参数	返回	解决哪类问题
read_dcs_tag	tag (位号)	值、单位、时间戳、质量位	时效性: 现在多少
query_property	substance, property, temperature	数值、单位、数据来源	查物性数据库, 免心算
unit_convert	value, from, to	换算后的值	单位换算, 零误差
get_sds	cas 或物质名	闪点、危险类别、应急措施	查安全数据表 (SDS)

再进一步, 还可以加: `mass_balance` (物料衡算)、`query_lims` (查化验数据)、`create_work_order` (生成巡检工单)。

每个函数都对应第一部分说的“四要素”; 名字用动词开头, 是为了让模型一眼看出它“做什么”。

动手写: 给 query_property 写函数声明

课堂互动④

4 人一组, 5 分钟, 照着第三部分那份 read_dcs_tag 的样子, 在纸上为“查物性数据库”写出 JSON Schema:

name · description · 三个参数 (substance / property / temperature) 的类型与说明 · required。

写完互换检查, 用这三个问题给隔壁组打分:

1. 只看你们的 description, 模型能判断“苯在 40 度的蒸气压”该调这个函数吗?
2. temperature 的单位写清楚了吗? 是 °C 还是 K? 没写清, 模型会怎么猜?
3. property 允许哪些取值 (蒸气压 / 密度 / 黏度...)? 要不要用 enum 限定死?

函数声明写得含糊, 模型调错的概率就直线上升——这一页和第三讲的 Prompt 是同一件事。

description 是写给模型看的 Prompt: 好坏对比

✗ 含糊的写法

```
"name": "query",  
"description": " 查数据",  
"temperature": {"type":  
  "number"}
```

模型不知道“查数据”和“读 DCS”有何区别; 温度单位靠猜; 用户说“40度”时可能传成 40 K。

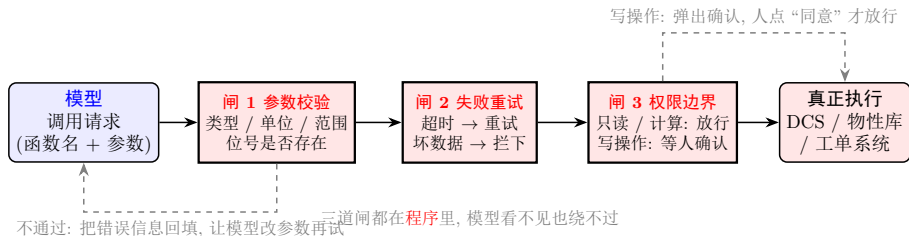
✓ 清楚的写法

```
"name": "query_property",  
"description": " 查询纯物质在指定  
温度  
下的物性 (蒸气压/密度/黏度),  
数据来自 NIST 库",  
"temperature": {"type":  
  "number",  
  "description": " 温度, 单位摄氏  
度"}
```

什么时候调、参数什么单位、数据从哪来, 一目了然。

三条经验: 说清**适用场景**; 参数写明**单位与格式**; 取值有限的用 `enum` 列出来。

程序这一侧的三道闸：校验、重试、权限



模型生成的参数是**猜出来的**, 不是查出来的: 位号可能不存在, 单位可能写反, 温度可能超出数据库范围。

每一个调用请求,在真正执行之前都要过这三道闸——这是程序的责任,不能指望模型自觉。

第一道闸：参数校验——错了就“退回去重写”

模型可能犯的错	例子	程序怎么办
位号不存在	把“R-301 入口温度”猜成 TI-310	查位号表；不存在就回填“位号无效，候选：TI-301, TI-302”
类型或单位不对	temperature: "40度" (应为数字)	按 Schema 校验类型；不通过就回填错误信息
超出范围	查苯在 800 °C 的液体密度	检查物理合理性；返回“超出数据库范围”
漏掉必填项	只给了 substance, 没给 property	required 检查；让模型向用户追问

关键做法：校验失败时，**不要直接报错给用户**，而是把错误信息当作函数结果回填给模型，让它**修正参数再试一次**。

这和实验室里的 SOP 一样：试剂标签对不上，先核对再动手，而不是凭印象加。

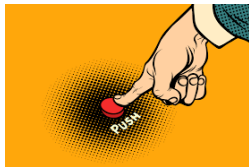
第二道门：失败重试与降级——外部世界不总是在线

接口超时	DCS 网关 3 秒没响应	重试 2 次, 间隔递增; 仍失败则告知“暂时读不到”
返回异常值	质量位 BAD, 或值为 -9999	不把坏数据交给模型; 回填“该位号数据无效”
查不到	物性库里没有这种混合物	回填“未收录”, 让模型如实告诉用户, 而不是编一个数
结果太大	一次拉回 5 万行历史数据	在程序里先聚合 (均值 / 最大 / 最小), 只回填摘要

原则: 函数可以失败, 但失败必须被看见。“没查到”是一个合格的答案, “编一个”不是。

Q: 如果重试 3 次都失败, 模型会不会“自己想一个”温度填上去? 靠什么防住它?

第三道闸：权限边界——读、算、写，三个等级



有些按钮，不能让模型
直接按

等级	例子	规则
只读	读 DCS、查物性、 查 SDS	模型可自动调用，但只能读 有权限 的位号
计算	单位换算、物料衡算、跑一次模拟	自动调用；结果标明来源与假设
写操作	生成工单、发邮件、 写回 LIMS	模型可 提议 ，执行前必须 人确认
禁止	修改设定值、开关 阀门	根本不注册 给模型：没有这个函数，就不可能被调

最可靠的权限控制不是“告诉模型别做”，而是**不给它那把钥匙**。

小组讨论: 给 8 个函数分等级

课堂互动⑤

4 人一组, 3 分钟。把下面 8 个函数分到“自动调用 / 需人确认 / 不注册”三档, 每组报一个**最有争议的**, 说明理由:

读位号温度 · 查物性 · 单位换算 · 导出班报表 · 给班组群发消息 · 生成巡检工单 · 修改 PID 设定值 · 打开泄压阀

可能引起争论的两个:

- ▶ **导出班报表**: 只是读数据, 但导出的文件会**离开系统**——是否算“写”? 谁能收到?
- ▶ **给班组群发消息**: 内容是模型写的, 发出去就收不回——一条写错的“降温指令”后果是什么?

分级的依据不是“技术上能不能”, 而是“做错了能不能撤回、损失有多大”。

第五部分 · Function Call 与 MCP、Agent 的关系

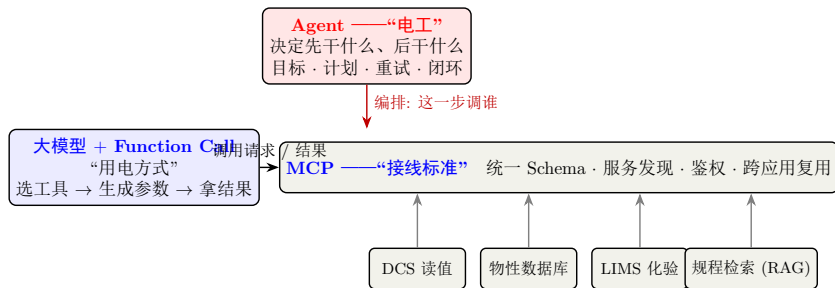
有了 Function Call, 模型能“**调一次**工具”。但真实业务往往不是一次就完:

- | | |
|-----------|-----------------------------------|
| 先调谁、怎么排顺序 | 先读温度还是先查上限? 失败了要不要换条路? |
| 失败如何重试 | 谁来决定重试几次、什么时候放弃? |
| 工具如何统一接入 | DCS、LIMS、物性库各有各的接口, 每换一个模型就要重接一遍? |

于是长出两条演进线:

Agent 负责把多步流程编排成闭环; **MCP** 负责把工具接入标准化、可发现、可治理。

一个类比：用电方式、接线标准、电工



“插座”统一了，电器（工具）随便换；电工决定开哪个、按什么顺序开

Function Call	“用电方式”	描述模型如何调用一个工具：选工具 → 产出结构化参数 → 执行 → 回传
MCP	“接线标准”	描述工具如何被发现与接入：服务发现、能力声明、鉴权、跨应用复用
Agent	“电工”	决定先干什么后干什么：目标、计划、工具选择、执行逻辑

三者对比：一张表

维度	Function Call	MCP (Model Context Protocol)	Agent
作用	描述模型如何调用单个工具/函数	负责工具的发现、注册、统一接入	负责编排多步任务、形成执行闭环
位置	LLM 内部的一种调用行为	协议层，位于应用与工具之间	运行在应用层，调度 LLM 和 MCP
关键要素	函数名、参数、返回值	Schema 标准、服务发现、鉴权、复用	任务目标、计划、工具选择、执行逻辑
类比	“用电的方式”	“电网的接线标准”	“电工，决定先干什么后干什么”
应用场景	查询天气、查库存、下单	把企业/第三方 API 统一挂载	自动完成复杂业务流程（如订票+报销）

MCP 把“工具的定义与托管”从应用层挪到独立的 MCP Server: 按统一 Schema 暴露工具, 任何客户端连上就能自动发现并调用, 不用为每个模型重复接入。

Cursor、Dify、ChatGPT 等已能作为 MCP 客户端对接远程 MCP 服务器。

配图: Function Call / MCP / Agent 六个维度的对比。MCP $\neq$ “函数换了个地方放”, 它是协议层。

典型链路: 三者怎么配合

MCP 负责发现 / 接入 → 模型在客户端内用 Function Call 产出参数

→ 客户端调用该 MCP 工具, 把结果回传给模型 → Agent 决定下一步调谁, 直到目标达成

MCP Server	把工具摆到桌面上: 搜索、数据库、RAG 知识库、第三方 API
MCP Client	(ChatGPT、Cursor、Dify) 连上去, 把这些工具“挂载”到模型可见的环境
Agent	决定怎么用这些工具完成目标, 用 Function Call 机制一步步调用, 直到达成

工具可复用、跨应用接入, MCP Server 端可以按调用次数、按并发量收费——未来可能出现 “Agent-as-a-Service + MCP 工具市场”。

第六讲讲 MCP, 第七讲讲 Agent; 今天只需记住: 它们都建立在 Function Call 这块地基上。

素养与责任

让 AI 做事, 不等于让 AI 负责

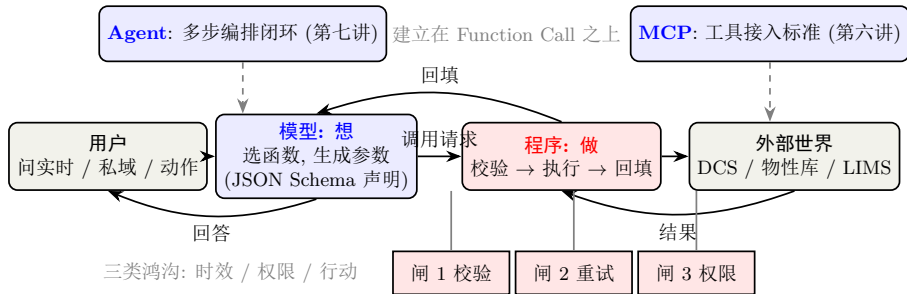
从今天起, 你写的每一个函数声明, 都是在给模型一把钥匙。四条具体做法:

- ▶ 写操作 (发消息、建工单、写数据库) 一律留“人确认”这一步, 哪怕它看起来很小;
- ▶ 回答里的每个数, 都要能追溯到“哪个函数、什么时刻、哪个来源”, 交作业、写报告时把这条链附上;
- ▶ 函数失败就如实说“没查到”, 不允许模型拿训练记忆补一个数;
- ▶ API 密钥、账号口令不写进 Prompt、不贴进聊天框: 钥匙交给程序保管, 不交给模型。

Q:讨论 (2 分钟): 如果你是值班工程师, AI 助手说“已根据温度自动下调了进料量”, 你的第一反应是什么? 这个系统哪里设计错了?

五个部分讲完, 下面收拢 $\Rightarrow$ 总结与快测

本讲总结: 一条回路, 三道闸, 两条演进线



模型只“想”: 选函数、生成参数、组织回答; 程序只“做”: 校验、执行、回填。
Function Call 补上时效性、权限、行动三道鸿沟; 往上长出 MCP (接入标准) 和 Agent (多步编排)。

六句话带走

它是什么	模型不自己知道答案, 而是调用外部函数取回结果, 再回答
为什么要	训练快照跨不过时效性、权限、行动三道鸿沟
模型做什么	只输出“调哪个、传什么参数”的 JSON, 一行代码都不执行
程序做什么	传函数声明、校验参数、真正执行、把结果回填
怎么注册	每次调用时用 JSON Schema 通过 <code>tools</code> 临时告诉模型, 不是永久写进去
怎么用得稳	校验、重试、权限三道闸; 写操作留人确认, 危险操作不注册

Q:回到开场中控室的三个问题: 现在你能说出每一个分别要调哪个函数、程序要在哪一步把关吗?

随堂快测

课堂互动 · 3 道题

- Q1 (单选)** 在 Function Call 回路里, 真正去读 DCS 数据的是谁?
A. 大模型 B. 应用程序 C. JSON Schema D. 用户
- Q2 (判断)** 把函数“注册”给模型后, 这个函数就永久存进了模型参数里, 以后不用再传。
- Q3 (简答)** 同学给助手加了 `set_valve_opening` (设置阀门开度) 函数, 并在 `description` 里写了“危险, 请勿随意调用”。用今天的内容指出问题, 并给出更稳妥的做法。

答案: Q1 选 **B**; Q2 **✗** 错误, 每次调用都通过 `tools` 临时传入, 不改参数; Q3 靠 `description` 劝阻不可靠——模型的输出是概率式的; 正确做法: 写操作不注册, 或执行前强制人工确认。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 写声明: 把课堂互动④的 `query_property` 整理成一份完整的 JSON Schema, 再补一个 `get_sds` (查安全数据表), 每个 `description` 不少于 20 字, 说明单位与取值范围。
- 必做② 读文档: 通读 [DeepSeek Function Calling 文档](#), 画出示例中“两次请求、一次执行”的时序图, 标出每一步是模型还是程序在做 (手绘即可)。
- 选做③ 跑起来: 在 [DeepSeek 开放平台](#)或 [阿里云百炼](#)申请 API, 把示例里的天气函数换成你的 `unit_convert`, 真的让模型调一次, 截图调用请求与回填结果。

拓展阅读: [百炼文档: 通义千问 Function Calling](#);
论文 [Toolformer \(2023\)](#): 模型如何自己学会用工具; [Gorilla \(2023\)](#): 让模型准确调用上千个 API (选读)。

下讲预告: MCP —— 让 AI 动起来的新“万能插座”

为什么每个工具都要接一遍

N 个模型 $\times$ M 个工具 = $N \times M$ 次接入, 受不了

MCP 是什么

一套统一的“接线标准”: Server 暴露工具, Client 自动发现

它和 Function Call 的分工

FC 管“怎么用电”, MCP 管“怎么接线”

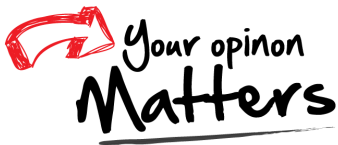
化工厂能用它做什么

把 DCS、LIMS、物性库做成 MCP Server, 谁来都能插

动手看一个 MCP Server

github.com/modelcontextprotocol
上的官方示例

Q:课前思考: 今天写的 `query_property`, 如果隔壁课题组也想用, 你要把哪些东西交给他们? 哪些不能交?



第五讲 · Function Call —— 让大模型实时感知与交互
周天航 · 中国石油大学 (北京)

下课。请完成课后任务, 下次课带上你写好的 `query_property` 函数声明。